MASSIVELY SCALABLE APPS LAB

Lab 2: Git Version Control

Fundamentals to Advanced Commands

PREPARED BY: MOHAMED AYMAN

Part A: Git Fundamentals

Version Control

Version Control System (VCS) = Software that tracks and manages changes to files over time

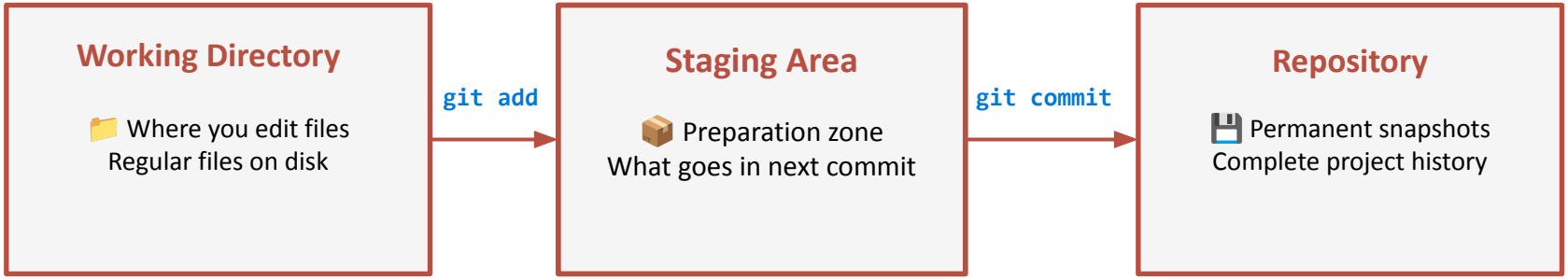
Key capabilities:

- Track complete history of every change
- Revert to any previous version
- Enable team collaboration
- See who changed what and when
- Protect against data loss

Think of it as: A time machine for your code that lets you travel back to any point in your project's history!

GIT'S Three-State Architecture

Git organizes work in three distinct areas:



Key Insight: The staging area gives you precise control over commits. You can choose exactly what changes to include!

Basic Git Workflow

Essential Commands

1. **Initialize repository:** `git init`
2. **Check status:** `git status`
3. **Stage files:** `git add <filename>` or `git add .`
4. **Commit changes:** `git commit -m "Your message"`

This three-step process (edit → stage → commit) is fundamental to Git!

Viewing History

```
# View commit history
git log

# Compact one-line format (most useful)
git log --oneline

# Visual graph of branches
git log --oneline --graph --all

# View unstaged changes
git diff

# View staged changes
git diff --staged

# View a specific commit
git show <commit-hash>
```

Tip: You only need the first 7 characters of a commit hash!

Using .gitignore

Prevent files from being tracked:

```
# OS files
.DS_Store
Thumbs.db

# IDE settings
.idea/
.vscode/

# Build artifacts
target/
dist/
*.class

# Dependencies
node_modules/
venv/

# Secrets (NEVER commit!)
.env
*.log
secrets.yml
```

Never commit: Passwords, API keys, tokens, large files, build artifacts, or IDE settings!

Part B: Branching & Merging

WHAT IS A BRANCH?

A branch = A pointer to a specific commit + a way to work on multiple versions simultaneously

Why branches matter:

🔄 Work on features without affecting main code

🔧 Fix bugs while feature development continues

👥 Multiple developers working independently

🧪 Experiment safely

Branches are lightweight! Creating a branch is just creating a 41-byte pointer—not copying your entire codebase.

ESSENTIAL BRANCH COMMANDS

```
# Create a branch
git branch feature-x

# Switch to a branch
git checkout feature-x

# Create AND switch (shortcut)
git checkout -b feature-x

# List all branches (* = current)
git branch

# Delete merged branch
git branch -d feature-x

# Force delete unmerged branch
git branch -D feature-x
```

Best Practice: Use descriptive names like feat/user-auth or fix/login-error, in general <type>/<ticket# and/or desc>

MERGING BRANCHES

Basic merge workflow:

```
# Switch to branch you want to merge INTO
git checkout main

# Merge the feature branch
git merge feature-x
```

Two types of merges:

- **Fast-Forward:** Main hasn't changed—just moves pointer forward
- **Three-Way:** Both branches have commits—creates merge commit

Merge Conflicts: When same lines/files edited in both branches. Git marks conflicts—you resolve manually, then git add and git commit.

Part C: Remote Repositories

REMOTE REPOSITORY BASICS

Remote = Copy of your repository on a server (GitHub, GitLab, BitBucket, etc.)

Why use remotes:

💾 **Backup:** Code survives laptop failure

👥 **Collaboration:** Team members share code

🌐 **Portfolio:** Showcase your work

🚀 **Deployment:** Production servers pull from remote

```
# Connect to remote
git remote add origin https://github.com/user/repo.git

# View remotes
git remote -v
```

PUSH AND PULL

Push = Send local commits to remote

```
# First push (sets upstream)
git push -u origin main

# Subsequent pushes
git push
```

Pull = Fetch remote commits + merge

```
# Get latest and merge
git pull origin main

# Or just
git pull
```

Best Practice: Always git pull before starting new work and before pushing!

CLONE AND FORK

Clone = Copy entire repository

```
git clone https://github.com/user/project.git
```

Fork (GitHub feature):

- Create your own copy on GitHub
- Make changes in your fork
- Create Pull Request to propose changes back

Pull Request workflow:

1. Push your branch to remote
2. Create PR on GitHub
3. Code review
4. Merge after approval

FORK VS CLONE

	Clone	Fork
What is it?	Copy repo to your local machine	Copy repo to your GitHub account
Where?	Local (your computer)	Remote (GitHub server)
Push access?	Only if you have permissions	Yes — it's YOUR copy
Connected to original?	Yes — same remote origin	No — separate repo (can add upstream)
Use case	Work on repos you own/have access	Contribute to others' projects

Use CLONE when:

- It's your own repository
- You're a collaborator with push access
- You're just downloading to work locally

Use FORK when:

- Contributing to open-source projects
- You don't have push access
- You want your own independent copy

```
# Clone (download to local)
git clone <url>
```

```
# Fork workflow
1. Click "Fork" on GitHub
2. git clone <your-fork-url>
3. Make changes, push to YOUR fork
4. Create Pull Request to original
```

Part D: Advanced Git Commands

WHY LEARN ADVANCED COMMANDS?

Real-world scenarios require more than basics:

🔧 Need to undo commits but keep changes?

🍒 Want to copy ONE commit to another branch?

📏 Need cleaner, linear history?

💾 Want to save work temporarily?

🔄 Need to recover deleted work?

Advanced commands give you surgical precision over your Git history and workflow!

`git stash`

GIT STASH - SAVE WORK TEMPORARILY

The Problem:

You're working on a feature, but need to switch branches urgently. Your changes aren't ready to commit.

The Solution:

```
# Save current changes to stash
git stash
# Your working directory is now clean
git status  # Shows nothing to commit
# Switch branches, do urgent work...
git checkout main
# ...fix bug, commit...
# Return and restore your work
git checkout feature-x
git stash pop
```

Stash = A hidden stack where you can temporarily store uncommitted changes

STASH COMMANDS

```
# Save with a descriptive message
git stash save "Work in progress on login"
# List all stashes

git stash list
# stash@{0}: On feature-x: Work in progress on login
# stash@{1}: On main: Debugging session

# Apply most recent stash (keeps it in stack)
git stash apply

# Apply AND remove from stack
git stash pop

# Apply specific stash
git stash apply stash@{1}

# Remove a stash without applying
git stash drop stash@{0}

# Clear all stashes
git stash clear
```

WHEN TO USE STASH

✅ Perfect for:

- Switching branches with uncommitted work
- Pulling updates when you have local changes
- Temporarily shelving experiments
- Quick context switches

❌ Not recommended for:

- Long-term storage (use branches instead)
- Sharing with others (stash is local only)

Tip: Use git stash save "message" to remember what each stash contains!

git reset

GIT RESET - UNDO COMMITS

Reset = Move HEAD (and optionally staging/working directory) to a different commit

Three modes:

Mode	HEAD	Staging	Working Dir
`--soft`	✓ Moves	✗ Unchanged	✗ Unchanged
`--mixed`	✓ Moves	✓ Reset	✗ Unchanged
`--hard`	✓ Moves	✓ Reset	✓ Reset

Warning: --hard is destructive! It permanently discards uncommitted changes.

GIT RESET --SOFT

Use case: Undo commit but keep all changes staged

```
# Undo last commit, keep changes staged
git reset --soft HEAD~1
```

Before:

```
A -- B -- C (HEAD)
           ↑
    Committed "oops, wrong message"
```

After:

```
A -- B (HEAD)

Changes from C are still staged, ready to recommit
```

Perfect for: Fixing commit messages, combining multiple commits into one

GIT RESET --MIXED (DEFAULT)

Use case: Undo commit AND unstage changes (keep in working directory)

```
# Undo last commit, unstage changes
git reset HEAD~1
# or explicitly:
git reset --mixed HEAD~1
```

Result:

- Commit is undone
- Changes are in working directory (modified files)
- Nothing is staged
- You can re-stage selectively

Perfect for: Breaking a commit into smaller commits, reviewing changes before re-committing

GIT RESET --HARD

Use case: Completely discard commits and all changes

```
# DANGER: Discard last 2 commits entirely
git reset --hard HEAD~2

# Discard ALL uncommitted changes
git reset --hard HEAD

# Reset to specific commit
git reset --hard abc1234
```

⚠️ DANGEROUS: This permanently deletes uncommitted work!
Always double-check before using --hard

When to use:

- Abandoning a failed experiment
- Clearing messed up working directory
- Reverting to known good state

RESET REFERENCE SYNTAX

HEAD~n = n commits before HEAD
HEAD^ = parent of HEAD (same as HEAD~1)

```
# Go back 1 commit
git reset HEAD~1

# Go back 3 commits
git reset HEAD~3

# Go to specific commit
git reset abc1234

# Go to where branch was yesterday
git reset HEAD@{1.day.ago}

# Unstage a file (without undoing commit)
git reset HEAD filename.txt
```

`git revert`

GIT REVERT - SAFE UNDO

Revert = Create a NEW commit to undo changes from a previous commit.

```
# Revert the last commit
git revert HEAD

# Revert a specific commit
git revert abc1234
```

Before:

```
A -- B -- C (HEAD)
           ↑
     "Add buggy feature"
```

After revert:

```
A -- B -- C -- C' (HEAD)
               ↑
     "Revert: Add buggy feature"
```

Key point: History is preserved! The original commit still exists.

RESET VS REVERT - CRITICAL DIFFERENCE!

Aspect	`git reset`	`git revert`
History	Rewrites/erases	Preserves (adds new)
Safety	Destructive	Safe
Shared branches	❌ Never use	✅ Safe to use

Golden Rule: Never reset commits that have been pushed! Use revert instead.

WHEN TO USE RESET VS REVERT

Use `git reset` when:

- Working locally (not pushed yet)
- You want to "erase" commits from history
- Cleaning up before pushing

Use `git revert` when:

- Commits are already pushed/shared
- You need to maintain history for audit
- Others might have pulled your commits
- Working on main/production branches

When in doubt, use revert! It's always safer.

`git cherry-pick`

GIT CHERRY-PICK - COPY SPECIFIC COMMITS

Cherry-pick = Apply changes from specific commit(s) to current branch

Scenario:
You made a bugfix on feature branch, but main needs it NOW (before feature is ready)

Solution:

```
main:    A -- B -- C

feature: A -- B -- D -- E -- F
                         ↑
                "Fix critical bug"
```

```
git checkout main
git cherry-pick E
```

```
main:    A -- B -- C -- E' (bug fixed!)

feature: A -- B -- D -- E -- F
```

CHERRY-PICK COMMANDS

```
# Cherry-pick a single commit
git cherry-pick abc1234

# Cherry-pick multiple commits
git cherry-pick abc1234 def5678

# Cherry-pick a range of commits
git cherry-pick abc1234..xyz9999

# Cherry-pick without committing (stage only)
git cherry-pick -n abc1234

# Abort if there are conflicts
git cherry-pick --abort

# Continue after resolving conflicts
git cherry-pick --continue
```

WHEN TO USE CHERRY-PICK

✅ Perfect for:

- Applying hotfixes to multiple branches
- Backporting features to older releases
- Recovering specific commits from abandoned branches
- Pulling specific changes without full merge

❌ Avoid when:

- You need ALL changes from a branch (use merge)
- The commit depends on previous commits
- You're cherry-picking frequently (consider better branching)

Note: Cherry-picked commits get NEW hashes! They're copies, not moves.

`git rebase`

GIT REBASE - REWRITE HISTORY

Rebase = Move your branch's commits to start from a different point

Before rebase:

```
main:     A -- B -- C -- D
               \
feature:        E -- F -- G
```

After `git checkout feature && git rebase main`**:**

```
main:     A -- B -- C -- D
                          \
feature:                   E' -- F' -- G'
```

Result: Feature branch now starts from latest main. Linear history!

HOW REBASE WORKS

Step by step:

1. Git finds common ancestor of branches
2. Git "removes" your commits temporarily
3. Git fast-forwards to target branch
4. Git replays your commits one by one
5. Commits get NEW hashes (they're recreated)

```
# Rebase feature onto main
git checkout feature
git rebase main

# If conflicts occur during rebase:
# Fix the conflict, then:
git add .
git rebase --continue

# Or abort entirely:
git rebase --abort
```

MERGE VS REBASE - THE GREAT DEBATE

Aspect	`git merge`	`git rebase`
History	Preserves (shows merge)	Linear (rewrites)
Commits	Original hashes kept	New hashes created
Conflicts	Resolve once	May resolve multiple times
Shared branches	✅ Safe	❌ Dangerous

WHEN TO USE MERGE VS REBASE

Use `merge` when:

- Combining feature into main/production
- Working with shared branches
- You want to preserve the exact history
- You're uncertain which to use

Use `rebase` when:

- Updating YOUR feature branch with latest main
- Cleaning up local commits before pushing
- You want linear, clean history
- Working on YOUR branch only

Golden Rule: Never rebase commits that have been pushed to shared branches!

MERGE VS REBASE - VISUAL COMPARISON

With Merge:

```
main:     A -- B -- C -- D -------- M
               \                    /
feature:        E -- F -- G -------
```

Shows when feature was merged (preserves context)

With Rebase:

```
main:     A -- B -- C -- D -- E' -- F' -- G'
```

Clean, linear history (looks like sequential work)

Neither is "better" — they serve different purposes!

Interactive Rebase

INTERACTIVE REBASE ACTIONS

Squash commits (combine multiple into one):

```
pick abc1234 Add login feature
squash def5678 Fix typo
squash jkl3456 More typo fixes
pick ghi9012 Add password validation
```

Result: 3 commits become 1 + password validation = 2 commits

Reword (change commit message):

```
reword abc1234 Add login feature  # Will prompt for new
message
```

Drop (remove a commit):

```
drop def5678 Fix typo  # Commit will be removed
```

Reorder (just change the order):

```
pick ghi9012 Add password validation
pick abc1234 Add login feature
```

WHEN TO USE INTERACTIVE REBASE

✅ Perfect for:

- **Squashing:** Combine "WIP", "fix typo", "oops" commits
- **Cleaning up:** Before creating a Pull Request
- **Reordering:** Group related changes together
- **Editing:** Split one commit into multiple
- **Fixing:** Change old commit messages

Pro tip: Clean up your commits with interactive rebase BEFORE pushing to make history easier to review!

GIT REBASE -I (INTERACTIVE)

Interactive rebase = Edit, reorder, squash, or drop commits

```
# Rebase last 4 commits interactively
git rebase -i HEAD~4
```

Opens editor with:

```
pick abc1234 Add login feature
pick def5678 Fix typo
pick ghi9012 Add password validation
pick jkl3456 More typo fixes

# Commands:
# p, pick = use commit
# r, reword = edit commit message
# e, edit = stop for amending
# s, squash = combine with previous
# f, fixup = like squash, discard message
# d, drop = remove commit
```

git squash

WHAT IS GIT SQUASH?

Squash = Combine multiple commits into a single, cleaner commit

Before Squash:

`WIP` → `Fix typo` → `Oops` → `Done!` → `Actually done`

⬇️ **squash**

After Squash:

✓ `Implement user authentication`

Result: Clean, professional commit history that tells a clear story!

HOW TO SQUASH COMMITS

Step 1: Start interactive rebase

```
# Squash last 5 commits
git rebase -i HEAD~5
```

Step 2: Mark commits to squash

```
pick abc1234 Add login feature      # Keep this one (first)
squash def5678 Fix typo              # Squash into previous
squash ghi9012 More fixes            # Squash into previous
squash jkl3456 Final cleanup         # Squash into previous
```

Step 3: Edit the combined commit message

```
# Git opens editor with all commit messages
# Write a single, meaningful message:
Implement user authentication

- Add login/logout functionality
- Include password validation
- Add session management
```

SQUASH VS FIXUP

Two ways to combine commits — choose based on what you want to keep:

Command	Keeps Message?	Use When
`squash (s)`	✓ Yes - combines all messages	You want to edit/combine messages
`fixup (f)`	✗ No - discards message	Quick fixes, typos (message not needed)

Example:

```
pick abc1234 Implement feature      # Main commit
fixup def5678 fix typo              # Discard this message
fixup ghi9012 oops forgot file       # Discard this message
squash jkl3456 Add error handling    # Keep this message
```

> Pro tip: Use fixup for quick fixes, squash when the commit message adds context!

SQUASH EXAMPLE - COMPLETE WORKFLOW

1. Your messy commit history:

```
$ git log --oneline
abc1234 Actually done now
def5678 Oops, forgot file
ghi9012 Fix bug
jkl3456 More WIP
mno7890 WIP
```

2. Run interactive rebase:

```
$ git rebase -i HEAD~5
```

3. Change pick to squash:

```
pick mno7890 WIP
s jkl3456 More WIP
s ghi9012 Fix bug
s def5678 Oops
s abc1234 Done
```

4. Write new commit message & save:

```
Implement user authentication

- Add login form
- Add password validation
- Add session management
```

5. Result - Clean history!

```
$ git log --oneline
xyz9999 Implement user authentication
```

> 5 messy commits → 1 clean, professional commit!

WHEN TO USE SQUASH

✓ PERFECT FOR:

- Before creating a Pull Request
- Combining "WIP" and "fix typo" commits
- Cleaning up feature branch history
- After code review changes
- Organizing commits by logical changes

✗ AVOID WHEN:

- Commits already pushed to shared branch
- Others have based work on your commits
- On main/master branch
- You need to preserve detailed history

Golden Rule: Only squash commits that have NOT been pushed to a shared branch!

BEST PRACTICE WORKFLOW:

Develop with many commits → **Squash before PR** → Push clean history

TROUBLESHOOTING SQUASH

Problem 1: Squash conflicts

```
# If conflicts occur during squash:
git status                    # See conflicting files
# Fix conflicts manually, then:
git add .
git rebase --continue
```

Problem 2: Want to abort squash

```
# Cancel and go back to original state:
git rebase --abort
```

Problem 3: Already pushed commits

```
# After squashing pushed commits (ONLY if no one else
pulled!):
git push --force-with-lease origin feature-branch

# WARNING: This rewrites remote history!
```

Always use --force-with-lease instead of --force. It prevents overwriting others' work!

GIT SQUASH - QUICK REFERENCE

Action	Command
Squash last N commits	`git rebase -i HEAD~N`
Squash down to specific commit	`git rebase -i <commit-hash>`
Abort rebase	`git rebase --abort`
Continue after fixing conflicts	`git rebase --continue`
Force push after squash	`git push --force-with-lease`

INTERACTIVE REBASE KEYWORDS:

```
pick (p)   = Use commit as-is
squash (s) = Combine with previous, keep message
fixup (f)  = Combine with previous, discard message
reword (r) = Use commit, but edit message
drop (d)   = Remove commit entirely
edit (e)   = Stop for amending
```

`git reflog`

GIT REFLOG - YOUR SAFETY NET

Reflog = Log of everywhere HEAD has pointed (local only)

```
git reflog
```

Output:

```
abc1234 HEAD@{0}: commit: Add feature
def5678 HEAD@{1}: rebase: finish
ghi9012 HEAD@{2}: rebase: start
jkl3456 HEAD@{3}: reset: moving to HEAD~2
mno7890 HEAD@{4}: commit: Deleted by reset!
```

Amazing fact: Reflog tracks everything—even commits "deleted" by reset! It's your time machine.

RECOVERING "LOST" COMMITS WITH REFLOG

Disaster scenario:

```
# Oops! Accidentally deleted commits
git reset --hard HEAD~3  # Lost 3 commits!
```

Recovery:

```
# Find the lost commit
git reflog
# abc1234 HEAD@{0}: reset: moving to HEAD~3
# def5678 HEAD@{1}: commit: Important work!  <-- There it
is!

# Recover it!
git reset --hard def5678  # Or:
git checkout def5678      # Or:
git cherry-pick def5678
```

Note: Reflog entries expire (default 90 days). Don't wait too long to recover!

REFLOG RECOVERY SCENARIOS

Scenario 1: Recover from bad reset

```
git reset --hard HEAD@{2}  # Go back 2 operations
```

Scenario 2: Recover from bad rebase

```
git reflog
# Find commit before rebase
git reset --hard HEAD@{n}
```

Scenario 3: Find deleted branch's commits

```
git reflog
# Find last commit of deleted branch
git checkout -b recovered-branch abc1234
```

Remember: git reflog is your "undo" for almost any Git operation!

Command Comparison Summary

UNDO COMMANDS COMPARISON

Command	What It Does	Modifies History?
`reset --soft`	Undo commit, keep staged	Yes
`reset --mixed`	Undo commit, unstage	Yes
`reset --hard`	Undo commit, discard all	Yes ⚠️
`revert`	Create undo commit	No ✅

INTEGRATION COMMANDS COMPARISON

Command	Use Case	History Impact
`merge`	Combine branches	Creates merge commit
`rebase`	Move branch base	Rewrites history
`cherry-pick`	Copy specific commits	Creates new commits
`rebase -i`	Edit commit history	Rewrites history

Rule of thumb: Anything that "rewrites history" should only be used on unpushed commits!

WHICH COMMAND SHOULD I USE?

Need to undo commits?

- Pushed? → git revert
- Not pushed, keep changes? → git reset --soft
- Not pushed, discard changes? → git reset --hard

Need to integrate branches?

- Combining feature into main? → git merge
- Updating YOUR branch with main? → git rebase
- Need just ONE commit? → git cherry-pick

Need to clean up commits?

- Before pushing? → git rebase -i

Made a mistake?

- Check git reflog for recovery options

Summary & Best Practices

ADVANCED COMMANDS SUMMARY

```
# Stash - temporary storage
git stash / git stash pop

# Reset - undo commits (local only!)
git reset --soft/--mixed/--hard HEAD~n

# Revert - safe undo (creates new commit)
git revert <commit>

# Cherry-pick - copy specific commits
git cherry-pick <commit>

# Rebase - move branch base
git rebase main

# Interactive rebase - edit history
git rebase -i HEAD~n

# Reflog - recovery tool
git reflog
```

GOLDEN RULES FOR ADVANCED GIT

🔑 Safety Rules:

- Never rebase or reset pushed commits
- Always use revert for shared branches
- Always check reflog before panicking

✨ Best Practices:

- Clean up commits with rebase -i BEFORE pushing
- Use descriptive stash messages
- Prefer merge for integrating features into main
- Prefer rebase for updating YOUR feature branch

Master these commands and you'll handle ANY Git situation!

COMPLETE COMMANDS REFERENCE

```
# BASICS
git init / git clone / git status / git add / git commit

# BRANCHES
git branch / git checkout -b / git merge

# REMOTE
git push / git pull / git fetch

# ADVANCED UNDO
git reset --soft/--mixed/--hard HEAD~n
git revert <commit>

# ADVANCED INTEGRATION
git cherry-pick <commit>
git rebase <branch>
git rebase -i HEAD~n

# UTILITY
git stash / git stash pop
git reflog
git log --oneline --graph --all
```

REMOVING ALREADY TRACKED FILES

The Problem:

You committed `.env` file early in the project. Now you add it to `.gitignore`... but Git keeps tracking it!

⚠️ **Why?** .gitignore only prevents **untracked** files from being added. Files already tracked are NOT affected.

The Solution:

Use `git rm --cached` to remove the file from Git's index (staging area) while keeping it on disk.

```
# Step 1: Add file to .gitignore
echo ".env" >> .gitignore

# Step 2: Remove from Git index (keeps file on disk!)
git rm --cached .env

# Step 3: Commit the changes
git commit -m "Stop tracking .env file"

# For a folder (e.g., node_modules):
git rm -r --cached node_modules/
```

What happens:

BEFORE:
.env is tracked by Git
(changes show in git status)

↓ `git rm --cached`

AFTER:
.env still exists locally
but Git ignores it ✓

⚠️ Warning: The file remains in Git history! To fully remove secrets, you need git filter-branch or BFG Repo-Cleaner.

Extra: Pull Requests

WHAT IS A PULL REQUEST?

Pull Request (PR) = A request to merge your changes into another branch

Also called "Merge Request" (MR) in GitLab

The PR Workflow:

Create Branch — Make Changes — Push to Remote — Open PR — Code Review — Merge

Why Pull Requests Matter:

- ✓ Code Review — Catch bugs before they reach main
- ✓ Discussion — Collaborate and improve code quality
- ✓ Documentation — Track why changes were made
- ✓ CI/CD Integration — Run tests automatically before merge

CREATING A PULL REQUEST

Step 1: Create and switch to a new branch

```
git checkout -b feature/add-login
```

Step 2: Make changes and commit

```
git add .
git commit -m "Add login functionality"
```

Step 3: Push branch to remote

```
git push -u origin feature/add-login
```

Step 4: Open Pull Request on GitHub

- Go to your repository on GitHub
- Click "Compare & pull request" button (appears after push)
- Or: Click "Pull requests" tab → "New pull request"
- Select base branch (main) and compare branch (your feature)
- Fill in title and description → Click "Create pull request"

PR TITLE & DESCRIPTION

❌ Bad PR:

```
Title: "Fixed stuff"

Description: (empty)
```

✓ Good PR:

```
Title: "Add JWT authentication"

Description: Implements user
authentication using JWT tokens...
```

PR Description Template:

```
## What does this PR do?
Adds JWT-based authentication for API endpoints.

## Why is this change needed?
Users need secure login functionality (Issue #42).

## How to test?
1. Run `npm test`
2. Try POST /api/login with valid credentials

## Screenshots (if UI changes)
[Attach screenshots here]

## Checklist
- [x] Tests added
- [x] Documentation updated
- [ ] Ready for review
```

CODE REVIEW BEST PRACTICES

FOR PR AUTHORS:

- Keep PRs small (< 400 lines ideal)
- Self-review before requesting
- Write clear commit messages
- Respond to feedback promptly
- Don't take criticism personally
- Squash commits before merge

FOR REVIEWERS:

- Be constructive, not critical
- Explain WHY, not just what
- Suggest improvements with examples
- Approve when "good enough"
- Review within 24 hours

Review Comment Examples:

❌ "This is wrong."

✓ "Consider using StringBuilder here for better performance in loops."

Goal: Improve code quality while maintaining team morale. Be kind!

BRANCH PROTECTION RULES

Branch Protection = Rules that prevent direct pushes and enforce code review

❌ WITHOUT Protection:

- Anyone can push directly to main
- No code review required
- Bugs go straight to production
- No audit trail
- Accidents happen easily

✓ WITH Protection:

- All changes go through PRs
- Required code reviews
- CI tests must pass
- Complete audit trail
- Mistakes caught early

Protected Branch Workflow:

Developer Pushes — Create PR — CI Tests Run — Code Review — Approved & Merged

Branch protection ensures code quality and prevents accidents on critical branches.

SETTING UP BRANCH PROTECTION

GitHub Settings Path:

```
Repository → Settings → Branches → Add branch protection rule
```

Step-by-Step:

1. Go to your repository on GitHub
2. Click "Settings" tab (need admin access)
3. Click "Branches" in left sidebar
4. Under "Branch protection rules", click "Add rule"
5. Enter branch name pattern: main (or master)
6. Configure protection options (see next slide)
7. Click "Create" to save the rule

> Tip: Use "main" or "master" to protect the default branch. Use "*" patterns for multiple branches.

PROTECTION OPTIONS EXPLAINED

Option	What It Does	Recommended
Require pull request before merging	No direct pushes allowed	✓ Yes
Require approvals	Set minimum reviewers (1-6)	✓ Yes (1-2)
Dismiss stale reviews	New commits invalidate old approvals	✓ Yes
Require status checks	CI tests must pass before merge	✓ Yes
Require branches up to date	Must merge/rebase main first	✓ Yes
Require signed commits	GPG signature verification	Optional
Include administrators	Rules apply to admins too	✓ Yes

> Start with basic protection (require PR + 1 approval + CI checks), then add more as needed.

RECOMMENDED CONFIGURATION

Small Teams (2-5 developers):

- ☑ Require pull request
- ☑ Require 1 approval
- ☑ Dismiss stale reviews
- ☑ Require status checks
- ☐ Require linear history
- ☑ Include administrators

Large Teams (10+ developers):

- ☑ Require pull request
- ☑ Require 2 approvals
- ☑ Dismiss stale reviews
- ☑ Require status checks
- ☑ Require branch up to date
- ☑ Require code owners review
- ☑ Include administrators

PR MERGE STRATEGIES

GitHub offers three merge options (configurable in Settings → General):

Strategy	Result	Best For
Create merge commit	Preserves all commits + adds merge commit	Full history tracking
Squash and merge	Combines all PR commits into one	Clean history (recommended)
Rebase and merge	Replays commits on top of base	Linear history, no merge commits

Visual Comparison:

```
Merge Commit:     A---B---C---M (main)
                       \     /
                        D---E (feature)

Squash & Merge:   A---B---C---S (main)     # S = D+E combined

Rebase & Merge:   A---B---C---D'---E' (main)  # Linear history
```

Recommendation: Use "Squash and merge" for clean main branch history with meaningful commits.

PR BEST PRACTICES SUMMARY

CREATING PRs:

- Keep PRs small and focused
- Write descriptive titles
- Use PR templates
- Link related issues
- Self-review before requesting
- Add screenshots for UI changes

REVIEWING PRs:

- Review within 24 hours
- Be constructive and kind
- Test the changes locally
- Check for edge cases
- Approve when good enough
- Use suggestions feature

REPOSITORY SETTINGS:

- ☑️ Enable branch protection on main/master
- ☑️ Require PR reviews (1-2 approvals)
- ☑️ Require status checks (CI/CD must pass)
- ☑️ Dismiss stale reviews on new commits
- ☑️ Use "Squash and merge" as default
- ☑️ Set up CODEOWNERS for automatic review requests

> PRs are not bureaucracy — they're a tool for collaboration, learning, and catching bugs early!

PULL REQUESTS - QUICK REFERENCE

GIT COMMANDS:

```
# Create feature branch
git checkout -b feature/my-feature

# Make changes, commit, push
git add . && git commit -m "Add feature"
git push -u origin feature/my-feature

# Update branch with latest main (before PR)
git fetch origin
git rebase origin/main

# After PR is merged, clean up
git checkout main
git pull origin main
git branch -d feature/my-feature
```

GITHUB SETTINGS:

Branch Protection:
Settings → Branches
→ Add rule

Merge Options:
Settings → General
→ Pull Requests

PR CHECKLIST:

□ Descriptive title □ Linked issue □ Tests pass □ Self-reviewed □ Updated docs □ Ready for review

ADDITIONAL RESOURCES

Interactive Learning:

🎮 Learn Git Branching: learngitbranching.js.org

🔀 Git Immersion: gitimmersion.com

Documentation:

📖 Git Book: git-scm.com/book

📚 GitHub Docs: docs.github.com

Cheat Sheets:

GitHub Git Cheat Sheet

Atlassian Git Tutorials

Best Practices:

Conventional Commits

Conventional Commit Messages

Semantic Versioning

Practice makes perfect! Create a test repository and experiment with these commands.

Thank You!

Questions?